

LivyLogs Full Setup Guide (Current Flow)

Very simple steps: do this, then this.

Read this first: real app entryptpoint

Start LivyLogs with `python livylogs.py`.

Parser/engine pieces still exist, but this guide uses the current top-level app startup flow.

Part 1: Start LivyLogs app

In PowerShell, run:

```
cd C:\Users\LivyC\PycharmProjects\livylogs
```

```
python --version
```

```
python .livylogs.py
```

Success looks like: the LivyLogs UI opens.

Part 2: First-run app checklist

- 1) Open Options window.
- 2) Select your SWG log file.
- 3) Confirm combat events begin updating.
- 4) Adjust overlay/opacity if wanted.

Success looks like: damage/healing/session values move live.

Part 3: App window map (what each one does)

- skimmers: quick high-level skim view
- damage_meter: live damage totals/rate
- leaderboard: ranking view
- details: deep per-player/event breakdown
- options: settings
- alexa: helper/AI window
- equalizer: audio/effect controls
- livius: tactical roster view
- gharv: utility analysis panel
- fax: utility/status panel
- discord_viewer: Discord link + relay state

Part 4: Uncle ReCoN (simple explanation)

Uncle ReCoN is loaded automatically during app startup.

You normally do not start it manually.

It supports intel/report-related workflows in the app ecosystem.

Part 5: Make Discord bot in Discord (one time)

- 1) Go to Discord Developer Portal and create a New Application.
- 2) Open the Bot tab and click Add Bot.

- 3) Turn ON Message Content Intent.
- 4) Copy the Bot Token (keep it secret).
- 5) In OAuth2 -> URL Generator: check 'bot' and 'applications.commands'.
- 6) Give it Send Messages + View Channel (+ Attach Files + Embed Links for advanced mode).
- 7) Open the invite URL and add the bot to your server.

Part 6: Run relay bot (basic Discord posting)

In PowerShell, run:

```
pip install discord.py aiohttp
$env:DISCORD_BOT_TOKEN="PASTE_YOUR_TOKEN_HERE"
$env:RELAY_PORT="8080" (optional)
python .\central_discord_bot.py
```

Success looks like: 'Relay API started on port 8080' and 'Logged in as ...'.

Part 6B: Public standalone bot (host once for everyone)

Like-I-am-5 version:

- You (bot owner) keep one bot running all the time on one host.
- Other people only invite your bot and link with one code.
- They do NOT run bot code on their own PC.

Owner setup:

- 1) Keep central_discord_bot.py online 24/7.
- 2) Put HTTPS in front (example: <https://relay.yourdomain.com>).
- 3) Tell users to set in PowerShell:
\$env:CENTRAL_BOT_API_URL="https://relay.yourdomain.com"

Success looks like: users only do /verify + VERIFY & LINK.

Part 7: Link LivyLogs with one code

- 1) In Discord, open the exact channel where you want messages.
- 2) Run /verify.
- 3) Discord shows a 6-character code (expires in 10 minutes).
- 4) In LivyLogs: open Discord Relay window.
- 5) Paste the code and click VERIFY & LINK.
- 6) You should see RELAY ACTIVE.

Part 8: Specific channel ID (optional but recommended)

If you want one exact channel:

- 1) Turn on Discord Developer Mode.
- 2) Right-click channel -> Copy Channel ID.
- 3) Run /verify in that exact channel.

Important: /verify binds to that exact guild_id + channel_id.

Part 9: Test relay mode

Do a short combat test in-game.

Success looks like: combat relay messages appear in the linked Discord channel.

Part 10: Advanced report + website mode (optional)

Create .env from .env.example, then fill:

- DISCORD_BOT_TOKEN
- DISCORD_CHANNEL_ID
- GITHUB_TOKEN
- GITHUB_REPO
- GITHUB_DOMAIN (optional)

Run:

```
pip install discord.py aiohttp python-dotenv cryptography pillow
python .\admin_tools\discord_bot_advanced.py
```

Success looks like: advanced bot logs in, tracks encounters, and can publish reports when GitHub values are valid.

Part 11: GitHub Pages setup (super simple)

Goal: put your HTML reports on a website with almost no stress.

- 1) Make a GitHub repo for reports.
- 2) In GitHub repo Settings -> Pages, turn on GitHub Pages.
- 3) In .env, fill:
 - GITHUB_TOKEN (can write to repo)
 - GITHUB_REPO (example: yourname/your-gh-pages-repo)
 - GITHUB_DOMAIN (only if using custom domain)
- 4) Run advanced bot again:

```
python .\admin_tools\discord_bot_advanced.py
```

Success looks like:

- Discord gets tactical report messages.
- GitHub gh-pages content updates.
- Website report URL opens.

Common oops fixes

- 'DISCORD_BOT_TOKEN not set': set the env var, then rerun the bot.
- 'DISCORD_CHANNEL_ID environment variable not set': add it to .env for advanced mode.
- 'Invalid or expired code': run /verify again and use the new code fast.
- 'Connection Error' in app: make sure central_discord_bot.py is running and reachable.
- No messages in channel: check bot permissions (Send Messages / View Channel).
- GitHub upload skipped: check GITHUB_TOKEN and GITHUB_REPO in .env.

Ready status checklist

- ☐ LivyLogs opens with `python .\livylogs.py`
- ☐ Options window set to your real SWG log file
- ☐ Bot invited to server
- ☐ `central_discord_bot.py` running
- ☐ (Public bot) Hosted URL is set with `CENTRAL_BOT_API_URL`
- ☐ `/verify` gives a code
- ☐ LivyLogs shows RELAY ACTIVE
- ☐ Combat pulse appears in Discord
- ☐ (Optional) HTML report published to GitHub Pages website